智慧幕墙石材幕墙污渍识别系统

设计规格说明文档

2251926 罗诗雨
2252092 毕雅琳
2251654 付宝莹
2252954 文彧婕

Category

1. Introduction 3
1.1 Objective 3
1.2 System Scope 3
2. System Overview 3
2.1 Background 3
2.2 Function 4
3. System Architecture 5
3.1 Architecture Design 5
3.2 Decomposition description 7
4. Interface Design 9
4.1 predict interface 9
4.2 history interface 10
5. Data Design 10
5.1 Database Design 10
5.2 Data Dictionary 10
5.3 Data Flow Diagram 11
6. Component Design 12
7. UI interface 13
7.1 Upload building curtain wall image 14
7.2 Start stain detection and view results 15
7.3 Download the inspection report 16
7.4 Query history 17
8. Appendix 19
8.1 Abbreviations and Glossary 19
8.2 Summary and reference documents 20

1. Introduction

1.1 Objective

The stain recognition system for stone curtain wall aims to automatically detect the stains on stone curtain walls in modern industry, providing assistance for engineering production and maintenance, enabling users to more effectively avoid safety risks and reduce economic costs. The system is based on the YOLOv8n model for image segmentation, non-local uniform noise reduction and image processing algorithms.

The purpose of this document is to provide a comprehensive overview of the software system architecture, component design, data design, interface design and User Interface of the stone curtain wall stain recognition system.

1.2 System Scope

The main customers and end users of the intelligent curtain wall system are the management of the civil engineering team of Tongji University. The project constructs a Web platform integrating functions such as stain recognition of stone curtain wall, crack recognition of stone curtain wall, image segmentation of glass curtain wall material, and 3D model of building. The goal is to provide intelligent analysis reports for civil engineering teams and other teams engaged in curtain wall design, construction, operation and maintenance, help them discover potential problems or risks in buildings, improve the accuracy and convenience of curtain wall structural stability and safety, and improve engineering efficiency.

The stain recognition system for stone curtain wall is a subsystem of the intelligent curtain wall system, which implements the image processing and recognition algorithms of the stain recognition module for stone curtain wall and part of the front-end interface, and realizes the interface connection between the front and back ends and the database. This subsystem is integrated into the complete intelligent curtain wall system through Docker.

2. System Overview

2.1 Background

(1) Application of curtain wall in modern architecture : As a part of the facade of modern architecture, curtain wall not only has beautiful visual effect, but also can play the functions of heat insulation, sound insulation and wind resistance. With the increase of high-rise buildings, commercial buildings and public buildings, the use of curtain wall has gradually become an important part of urban construction.

(2) The universality of curtain wall stain problems : Due to the weather, environmental pollution and the use of the building itself, the surface of the curtain wall is easy to accumulate stains, dust and pollutants, which not only affects the

appearance of the building, but also may affect the service life and function of the curtain wall (such as light transmittance, wind pressure resistance, etc.). Especially in some cities with poor air quality and serious industrial pollution, the cleaning problem of curtain walls becomes more prominent.

(3) Requirements for intelligent curtain wall stain recognition : Based on the above problems, an intelligent curtain wall stain recognition system came into being. The system combines Computer Vision and image processing. Through automatic detection and recognition, it can accurately determine the location and degree of stains on the curtain wall, and then provide scientific and reasonable cleaning solutions for cleaners. This not only greatly improves the cleaning efficiency, but also reduces the labor cost and safety risk. At the same time, it is of great significance to the long-term maintenance of the building.

2.2 Function

2.2.1 Image upload function

(1) Image upload: Users can upload pictures of the building facade through the system, supporting drag-and-drop upload/file selection upload and batch upload.

(2) Picture naming: After uploading, you need to name the picture for temporary storage and easy management.

(3) Management of the list of pictures to be detected: Pictures uploaded will automatically enter the "to be detected" list; support deletion or modification of uploaded pictures

(4) Select the image to be detected: Select a photo from the "To be detected" list as the image to be detected

2.2.2 Curtain wall stain detection function

(1) Curtain Wall Segmentation and Labeling : The system uses the trained YOLO model to detect and label curtain wall blocks. Based on the labeling results, the system segments the original image into one or more curtain wall block images.

(2) Stain detection : Perform stain recognition on each segmented curtain wall block image and return the detection results, including the stain area and stain proportion.

(3) Result visualization : The system returns the detection results to the front end for visualization.

2.2.3 Detection report export function

(1) Export the current inspection report: After the user successfully inspects a picture on the stain inspection page, he can click the "Export Report" button to export the inspection report.

(2) Export historical inspection reports: Users can click the "Export Report" button of

a certain inspection record on the history page to export the report.

2.2.4 Historical record viewing function

(1) Record management and archiving : The system saves the results of each stain detection as historical records, including information such as the time of detection and the detection results.

(2) Filter by time : Users can filter historical records by time period. This provides valuable data support for the long-term maintenance of buildings.

3. System Architecture

3.1 Architecture Design

This system is a curtain wall stain recognition system based on **front and back end separation** and **hierarchical architecture** . The front end uses the Nuxt3 framework to implement the user interface, and the back end provides business logic processing and API services based on the Flask framework. The entire system is integrated and deployed through containerization technology (Docker). The system data storage is divided into MySQL database and OSS object storage. The logical architecture and technical architecture are shown in the following two figures.

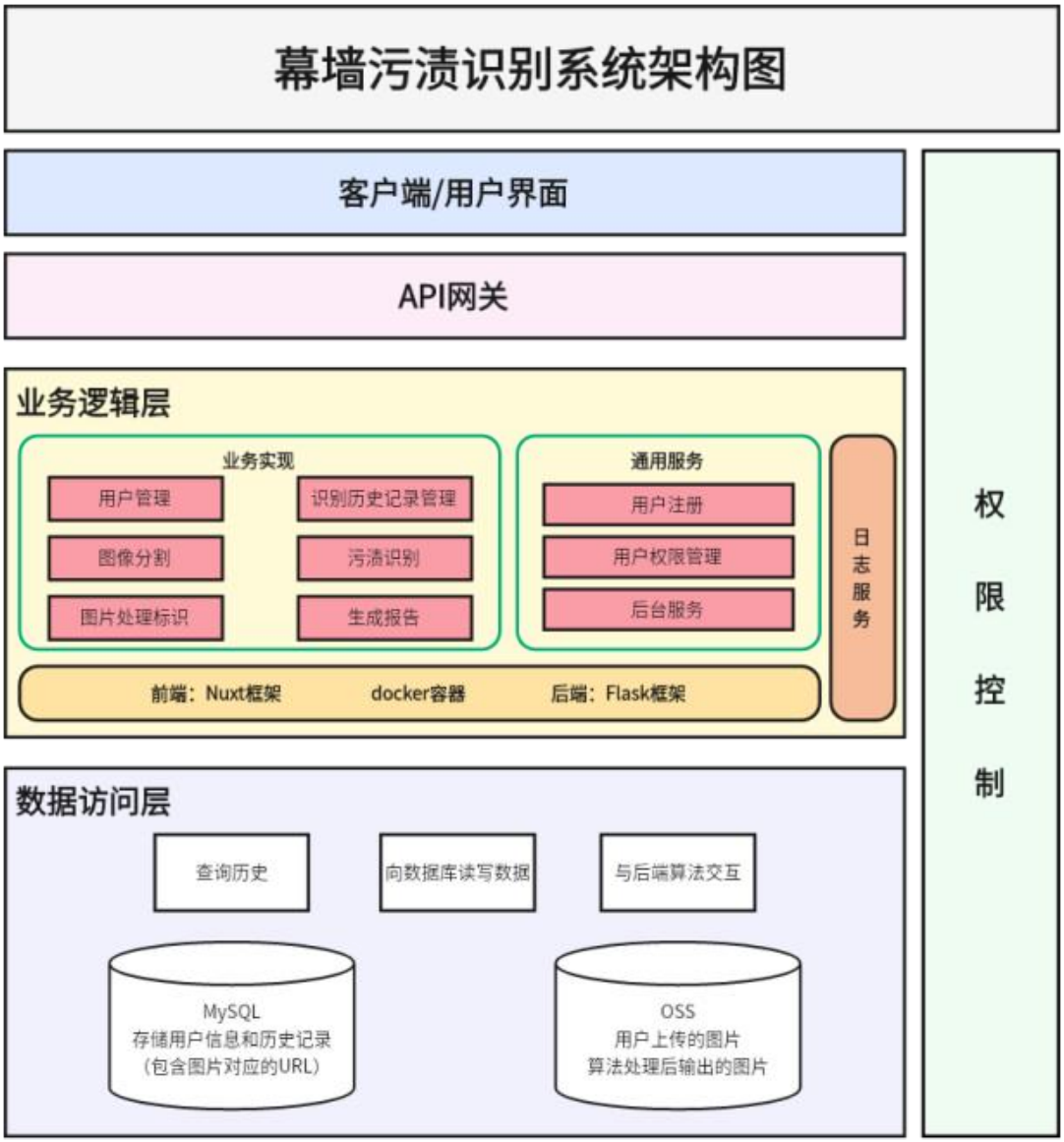


Figure 3-1 Logical Architecture Diagram

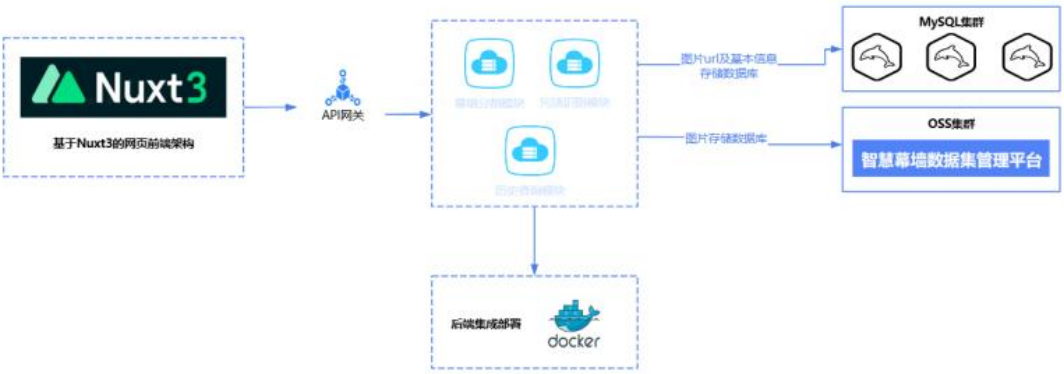


Figure 3-2 Technical Architecture Diagram

3.2 Decomposition description

3.2.1 System Architecture Diagram

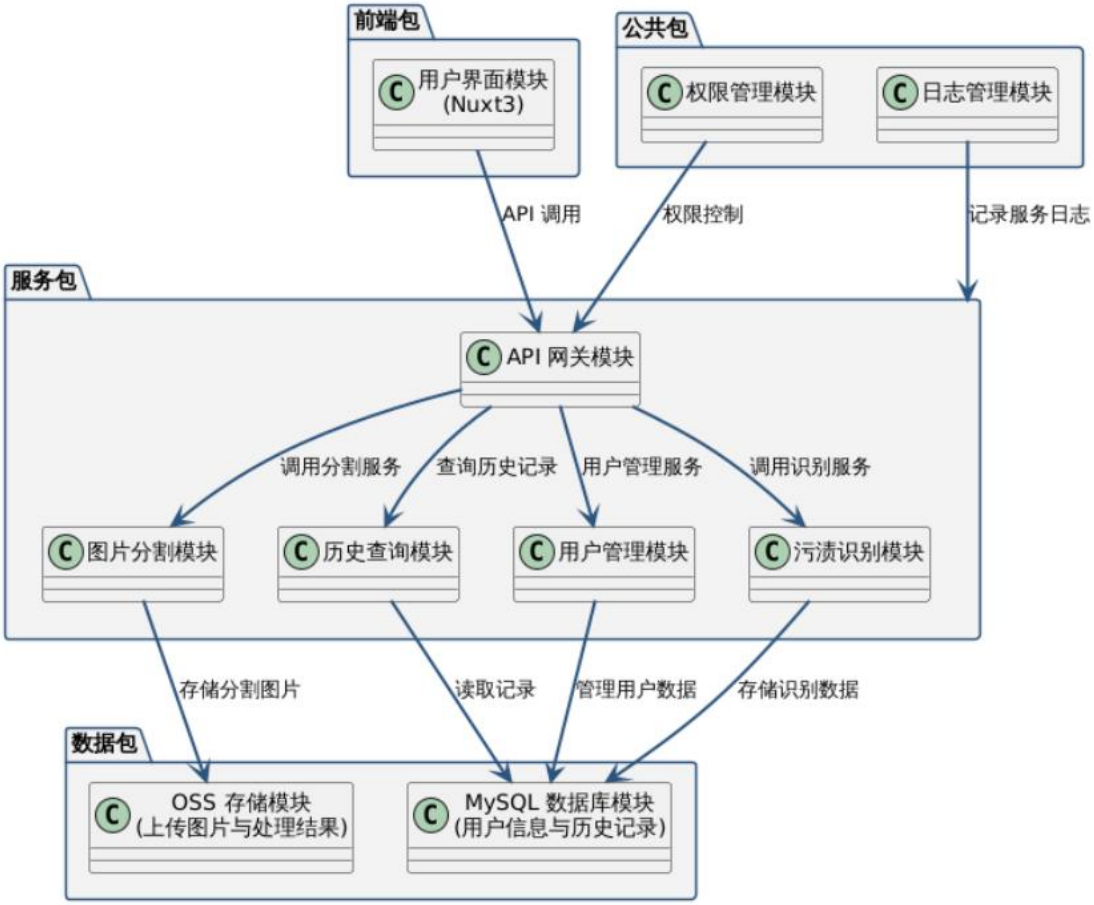


Figure 3-3 System Architecture Diagram

(1) Front-end package:

It includes a User Interface module (based on Nuxt3), which is responsible for interacting with users and making requests to the back-end API gateway module.

(2) Public package:

It includes the permission management module and the log management module.

• Rights management module: used to control the user's right of access and ensure that the user can only operate the function when he has the right.

• Log management module: Responsible for recording service logs to facilitate tracking of system operation.

(3) Service package:

This is the core part of the system, which connects the front - end requests and the back - end services through the API gateway module as the entry.

• Image segmentation module: Process the image segmentation function and store

the results in the OSS storage module.

• Historical query module: provides users with historical record query services and reads records from the MySQL database module.

• User management module: manages user information and permissions, and depends on the MySQL database module.

• Stain recognition module: Recognize stains in the picture and store the recognition data in the OSS storage module.

(4) Data packet:

It includes two major modules:

• OSS storage module: mainly responsible for storing pictures and processing results.

• MySQL database module: stores user information and historical records, supports service package query and data management requirements.

3.2.2 System function process

(1) Image upload and preprocessing:

1) Image upload

• The user uploads the target stone curtain wall image through the front-end interface.

• The system supports a variety of mainstream image formats (such as JPEG, PNG, etc.) to meet the diverse needs of users.

• The uploaded image will be transferred to the algorithm backend (Flask) via the web backend for processing.

2) Image preprocessing

The system performs preprocessing operations on the uploaded images, including:

• Resize: Unify the image resolution to ensure the consistency of the algorithm input.

• Contrast enhancement: Optimize the detail levels of the image to make the stain area clearer.

• Denoising: The non-local mean denoising algorithm is used to remove image noise, retain the key features of the image, and improve the accuracy of stain recognition.

(2) Stain recognition and segmentation:

1) Curtain Wall Division

• Use the YOLOv8 model to segment the image and accurately locate the area of the curtain wall block in the image.

• Improve the extraction accuracy of the target area and reduce background interference.

2) Stain recognition

• For the segmented curtain wall area, further stain recognition is carried out to mark the stain position.

(3) Stain area calculation:

• Calculate the area of each identified stain region to provide an accurate stain coverage.

• By segmenting the geometric data of the image, use mathematical methods (such as pixel area and actual scale conversion) to accurately measure the stain area.

(4) Result display and export:

The system displays the processed image results on the front - end interface:

• Clearly mark the stain area and visually present the recognition and segmentation results.

• The user can interactively view the detailed information of the stain, such as the stain location, area, etc.

Support the function of viewing historical results:

• Provide the function of viewing the images and data information of the user's historical stain recognition, so that the user can use the recognition results again.

4. Interface Design

API	Interface Introduction
POST /predict	Perform stain recognition function
POST /history	Get history

4.1 predict interface

- Request parameters: username, image_url
- Return parameter: results[]

none form-data x-www-form-urlencoded (•) json xml raw binary GraphQL msgpack

动态值 格式化

```
{
  "username": "2429638087@qq.com",
  "image_url": "http://110.42.214.164:9000/oss/download/stain-detection/upload/test2.jpg"
}
```

4.2 history interface

- Request parameters: username, start_time, end_time (can be null)
- Return parameter: data[]

Body Cookie Header 9 控制台 实际请求• 分享 200 7.42 s 722 B

Pretty JSON utf8

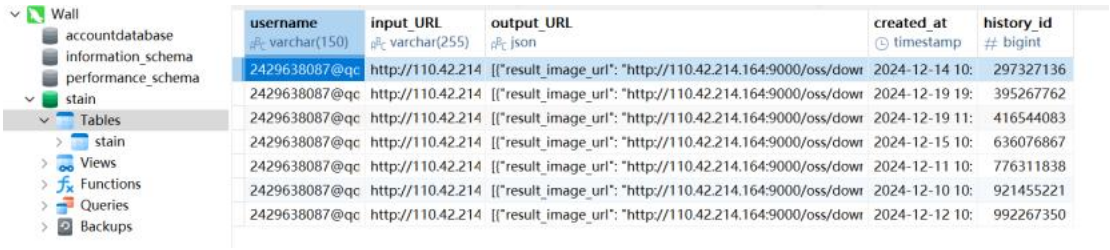

```
{
    "results": [
        {
            "result_image_url": "http://110.42.214.164:9000/oss/download/stain-detection/output/result-image-20241219114139-1.jpg",
            "stain_percentage": 0.27575959654242277,
            "warped_image_url": "http://110.42.214.164:9000/oss/download/stain-detection/output/warped-image-20241219114139-1.jpg"
        },
        {
            "result_image_url": "http://110.42.214.164:9000/oss/download/stain-detection/output/result-image-20241219114139-2.jpg",
            "stain_percentage": 0.40436161301902596,
            "warped_image_url": "http://110.42.214.
```


5. Data Design

5.1 Database Design

- Database type: MySQL database and OSS database
- Database function: MySQL database - store historical record information

OSS database - store pictures



username varchar(150)	input_URL varchar(255)	output_URL json	created_at timestamp	history_id bigint
2429638087@qc	http://110.42.214	[{"result_image_url": "http://110.42.214.164:9000/oss/dowr	2024-12-14 10:	297327136
2429638087@qc	http://110.42.214	[{"result_image_url": "http://110.42.214.164:9000/oss/dowr	2024-12-19 19:	395267762
2429638087@qc	http://110.42.214	[{"result_image_url": "http://110.42.214.164:9000/oss/dowr	2024-12-19 11:	416544083
2429638087@qc	http://110.42.214	[{"result_image_url": "http://110.42.214.164:9000/oss/dowr	2024-12-15 10:	636076867
2429638087@qc	http://110.42.214	[{"result_image_url": "http://110.42.214.164:9000/oss/dowr	2024-12-11 10:	776311838
2429638087@qc	http://110.42.214	[{"result_image_url": "http://110.42.214.164:9000/oss/dowr	2024-12-10 10:	921455221
2429638087@qc	http://110.42.214	[{"result_image_url": "http://110.42.214.164:9000/oss/dowr	2024-12-12 10:	992267350

5.2 Data Dictionary

Name	Type	Length
username	varchar	150
input_URL	varchar	255
output_URL	json	
created_at	timestamp	
history_id	bigint	

Database name: Stain

Table name: stain

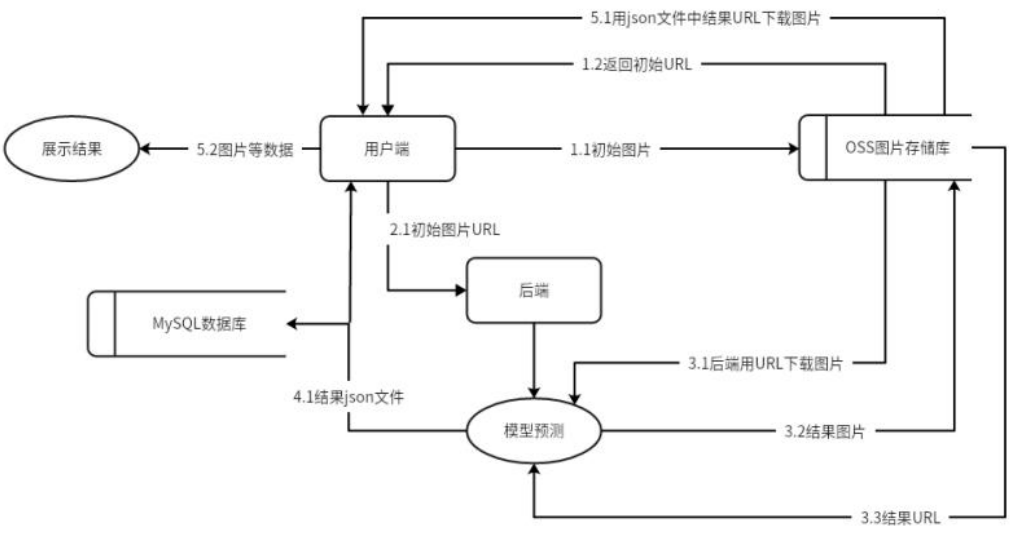
Table description: Store the detection history

Attributes:

- username: Username
- input_URL: The URL of the image to be detected entered by the user
- output_URL: json file of detection results, including the segmented image, the image of the dirty curtain wall (without stain annotation), the image of the dirty curtain wall (with stain annotation), and the stain percentage
- created_at: Detection time
- history_id: History record ID

5.3 Data Flow Diagram

5.3.1 Data flow diagram of stain recognition function



(1) The user selects a picture and uploads it to OSS, and the OSS library returns the URL

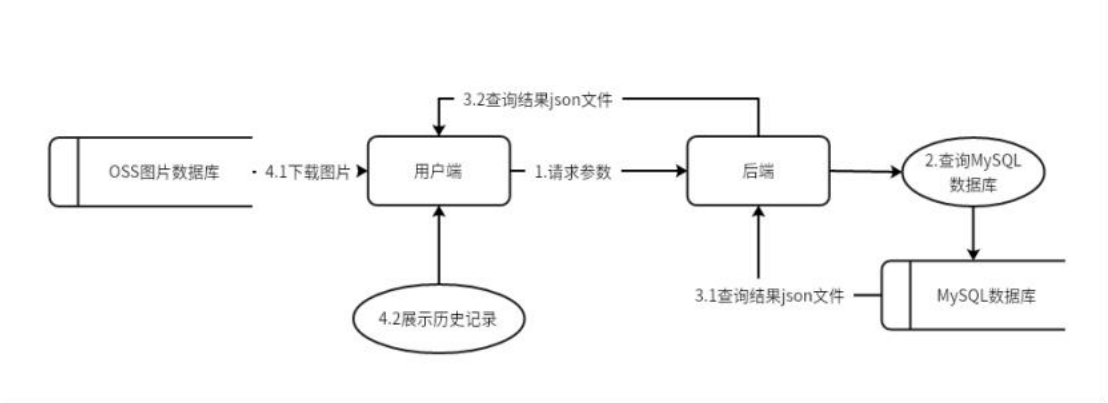
(2) After the user sends the URL to the backend, the backend uses the URL to download the image from the OSS library

(3) The back end performs the model prediction function, uploads the result picture to the OSS library, and returns the URL

(4) The back end stores JSON data such as the original picture URL and the result URL in the MySQL database and returns it to the front end at the same time.

(5) After the front end gets the return result, it parses and displays the JSON data.

5.3.2 Historical record query data flow diagram



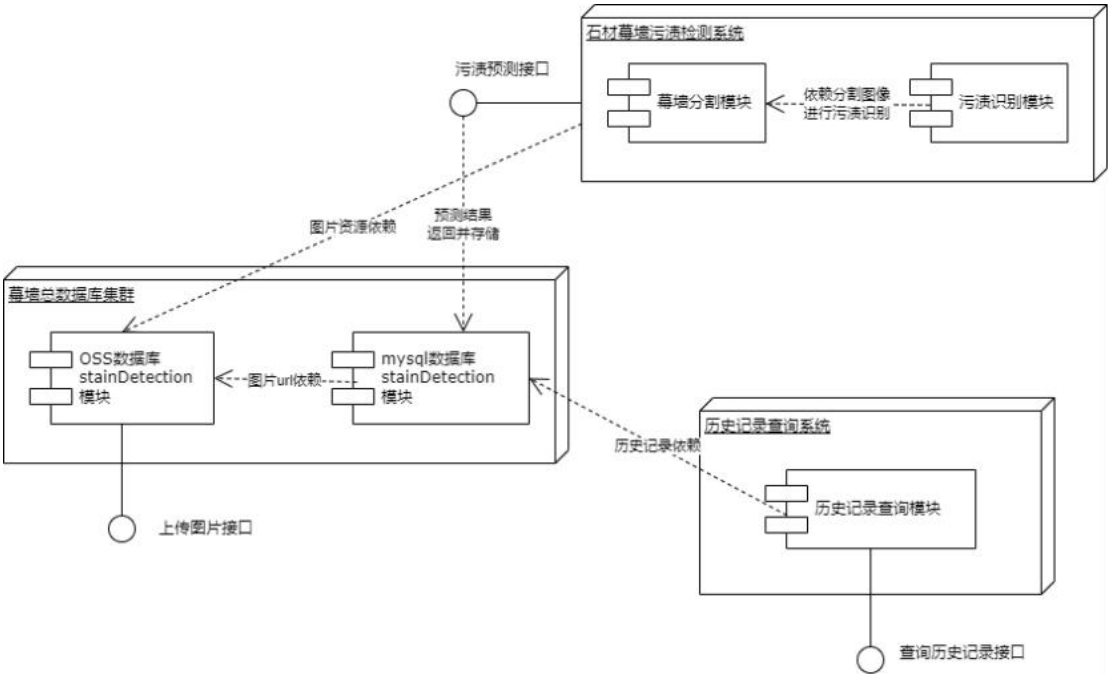
(1) The user sends a request, and the request parameters are passed to the back end

(2) The back end queries the data in the MySQL database

(3) The back end gets the JSON file of the query result and returns it to the front end

(4) The front end uses the URL in the JSON file to download the picture from the OSS library and displays the complete historical record.

6. Component Design



In the aspect of component design, our system is mainly divided into three parts: stone curtain wall detection system, historical record query system, curtain wall total database cluster system.

The stone curtain wall detection system includes a curtain wall segmentation module and a stain recognition module. The pictures of the stain recognition module depend on the result pictures of the curtain wall segmentation. The input pictures and result pictures of the stone curtain wall detection system need to be stored in the stainDetection module of the OSS database.

The historical record query system includes a historical record query module, and the queried historical records are stored in the stainDetection module of the mysql database.

The curtain wall total database cluster system includes the OSS database stainDetection module and the mysql database stainDetection module. The mysql database stainDetection module only stores basic information and the url of the picture. The url of the picture depends on the return result of the OSS database stainDetection module.

The stone curtain wall detection system provides an interface for stain prediction and stores the prediction results in the stainDetection module of the mysql database. The historical record query system provides an interface for querying historical records to query the results of stain recognition. The curtain wall total database cluster system provides an interface for uploading pictures and stores the input pictures in the stainDetection module of the OSS database.

7. UI interface

7.1 Upload building curtain wall image

1. Select the upload method

Users can drag image files into the upload area or click the "Upload Image" button to select them.

2. Customize the image name

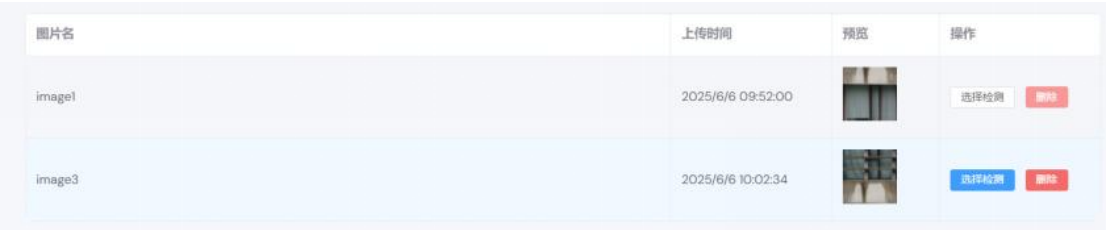
The system provides a text box to name each picture before or during uploading, which is convenient for subsequent identification and management.

3. The picture was uploaded successfully

The successfully uploaded images will automatically enter the "to be detected" list.

4. Manage uploaded images

You can delete the undetected pictures in the "to be detected" list.



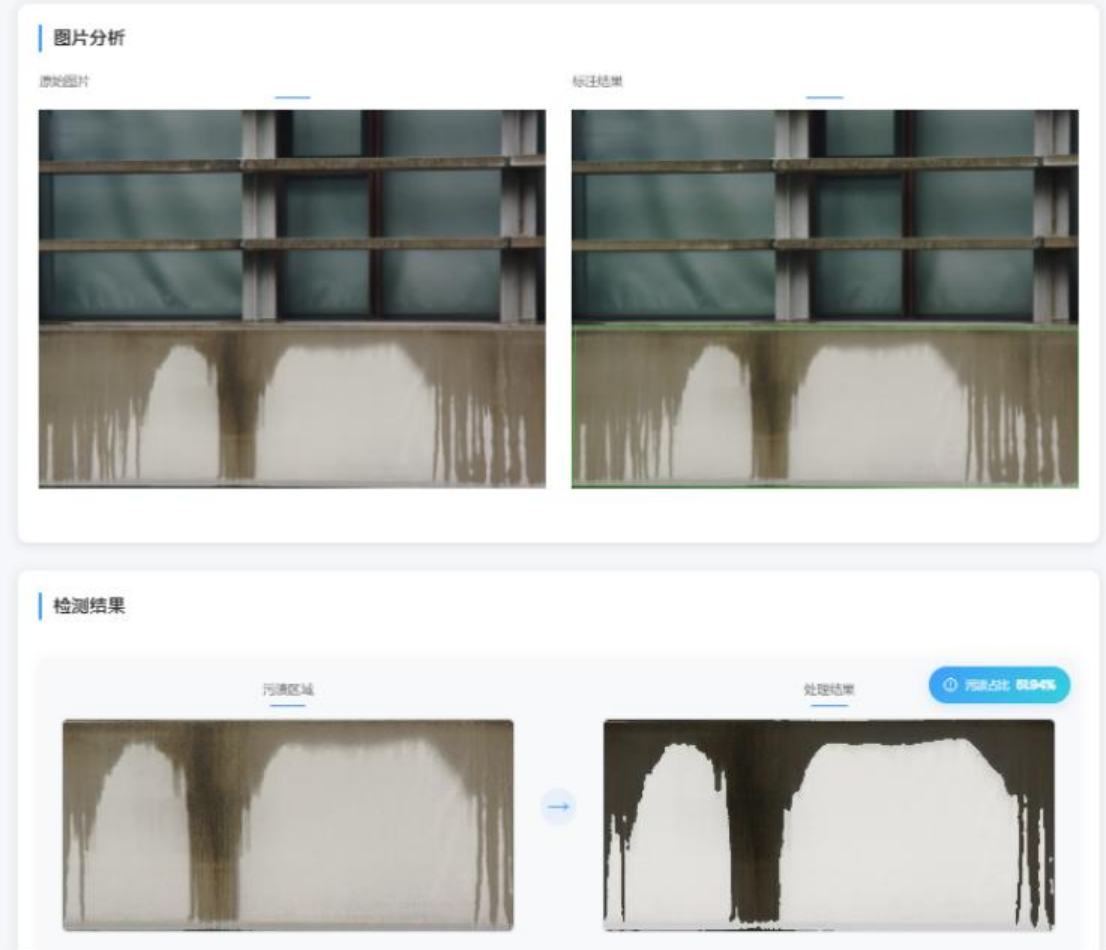
Notes: JPG and PNG formats are supported. It is recommended that the file size does not exceed 10MB, and the maximum support is 50MB.

7.2 Start stain detection and view results

1. Click the picture to start the detection

Click the specified picture in the "to be detected" list, and click "start detection" to identify stains.

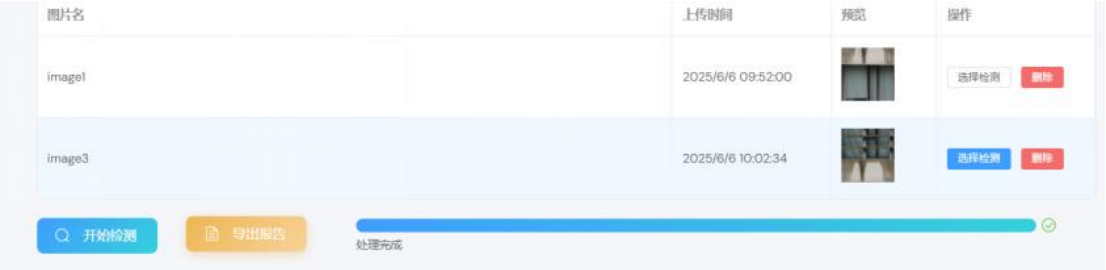
2. Check the detection results



7.3 Download the inspection report

1. Export Report

After the detection is completed, click the "Export Report" button, and the system will automatically generate a report file.



2. Report Format and Content

Export to PDF for easy printing or long-term storage.

污渍检测报告

一、用户信息

用户名：3041739348@qq.com

生成时间：2025/6/6 10:38:22

二、图片信息

图片名称：image3

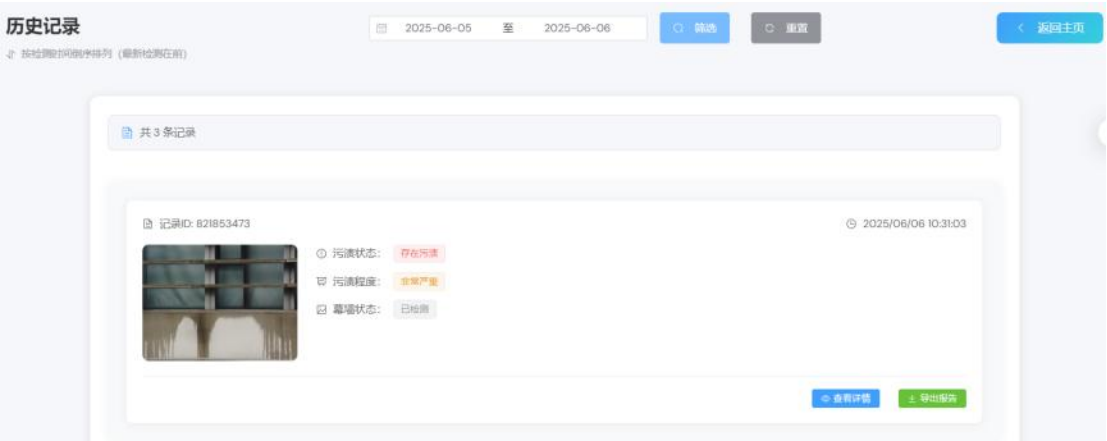
上传时间：2025/6/6 10:02:34

图片路径：http://110.42.214.164:9000/oss/download/stain-detection/upload/1749175352898-image3

7.4 Query history

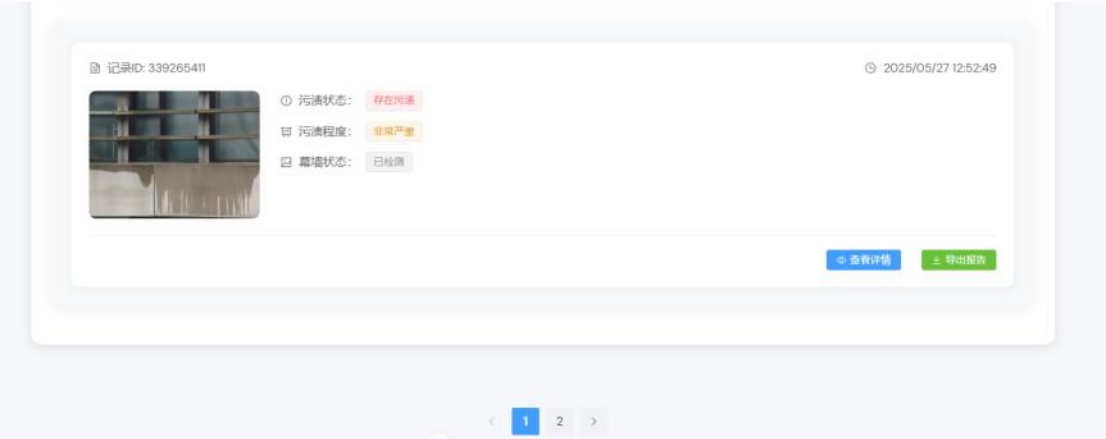
1. Filter records

You can set the start and end time to view the detection records only within the specified time period.



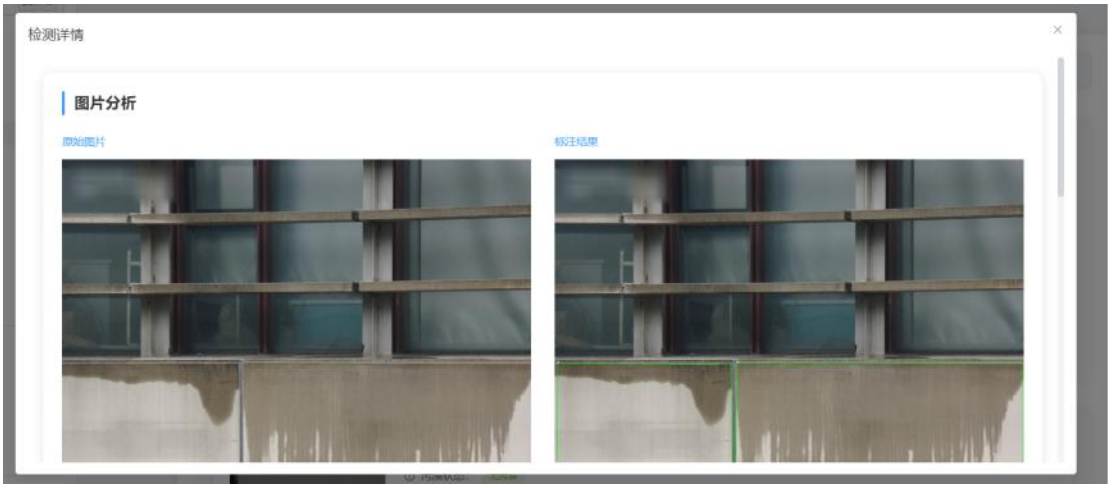
2. Record Sorting and Pagination

The historical records are arranged in descending order of detection time (the latest first) and displayed in pages to improve browsing efficiency.



3. View Details

Select a record to view the detailed inspection results



8. Appendix

8.1 Abbreviations and Glossary

Abbreviations / Terms	Full name/Explanation
YOLO	You Only Look Once, a real-time object detection system
CNN	Convolutional Neural Network, Convolutional Neural Network
API	Application Programming Interface, API
RESTful	Representational State Transfer, an HTTP-based API design style
JSON	JavaScript Object Notation, a lightweight data exchange format
JPEG	Joint Photographic Experts Group, a common image format
PNG	Portable Network Graphics, a lossless compressed image format
Flask	A lightweight Web application framework written in Python
SpringBoot	Tools for simplified configuration and deployment based on the Spring framework

Nuxt	A framework based on Vue.js that supports server - side rendering (SSR) and static site generation (SSG) for building modern websites or applications.
Docker	A containerized platform that packages applications and their dependencies into lightweight containers to ensure consistency across development, testing, and production environments.
MySQL	A relational database management system
Non-local uniform noise reduction	An image denoising algorithm that denoises by considering the non - local pixel similarity of the image
URL	Uniform Resource Locator, Uniform Resource Locator

8.2 Summary and reference documents

8.2.1 Group work summary document

Steps for Using the OSS Library.docx

Backend Integration Steps and Notes.docx

8.2.2 Reference Documents

Git Development Specification and Process for Backend Application Integration of Smart Curtain Wall Project.pdf(ljs)

Database Usage Documentation.docx

Smart curtain wall front end specification.docx